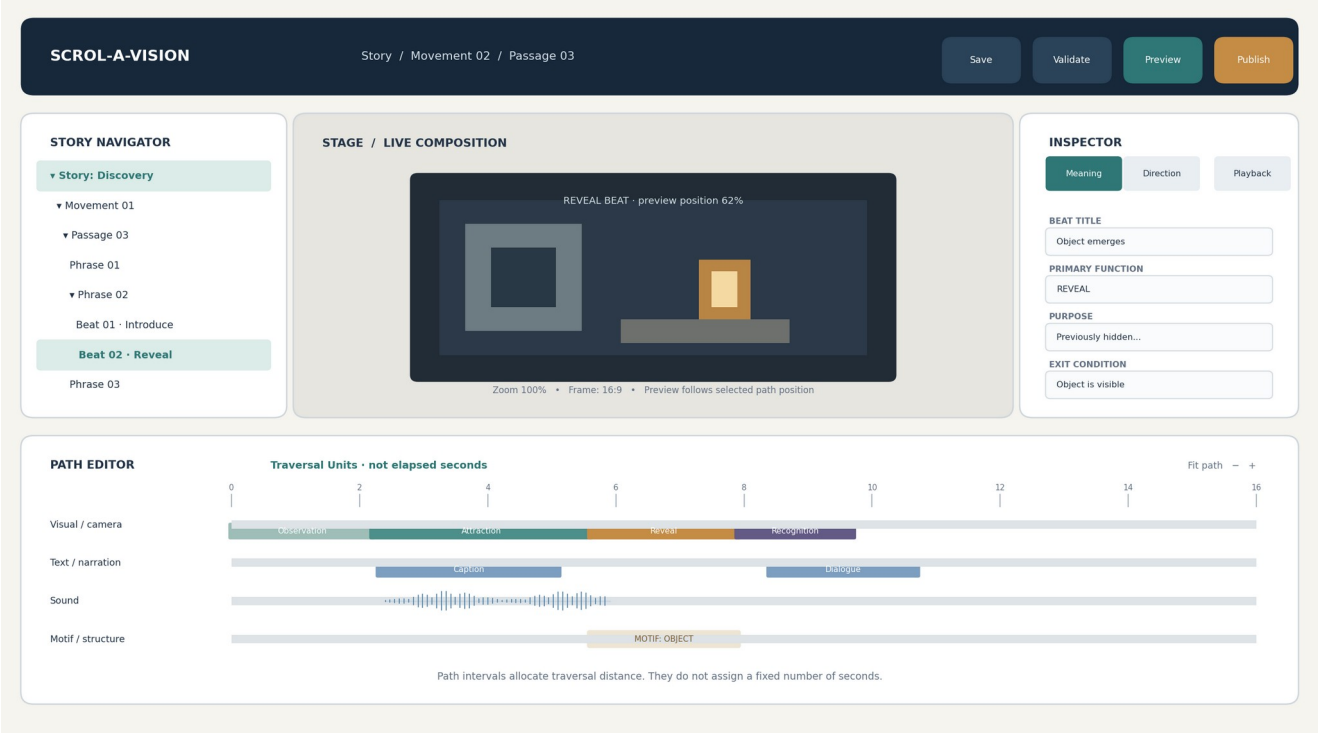
SCROL-A-VISION

STORY FORMAT & PLAYBACK

IMPLEMENTATION BLUEPRINT

DOCUMENT B / THE SOFTWARE CONTRACT

An implementation blueprint for an authoring environment and reader runtime built around the Scroll-Driven Storytelling Grammar.



Conceptual Studio layout: a proposed workspace, not a pixel-level UI mockup.

VERSION 1.0 ● OCTOBER 2026

Companion to Document A: Scroll-Driven Storytelling Grammar — The Creative Standard, v1.2

Document control

Field	Definition
Document	B — Story Format and Playback Implementation Blueprint
Companion standard	Document A: Scroll-Driven Storytelling Grammar — The Creative Standard, v1.2
Status	Proposed implementation contract for Scrol-A-Vision; test and refine against authored works
Primary audience	Product designer, frontend engineer, creative technologist, writer/director, visual artist, sound designer, QA
Baseline application	Existing React + TypeScript + Vite Studio/Player prototype; extend incrementally, do not restart without evidence
Core technical principle	One canonical story model; multiple coordinated views; deterministic output from authored path position

How to use this blueprint

Document A defines the creative meanings. This document specifies how Scrol-A-Vision should represent, edit, validate, save, resolve and publish those meanings. When a technical proposal conflicts with Document A, treat the conflict as a design issue to resolve explicitly; do not silently redefine the creative term inside the code.

The blueprint distinguishes: (1) requirements inherited from the creative standard, (2) proposed product decisions, and (3) implementation details that may evolve without changing the grammar. It is not a claim that every proposed feature already exists in the prototype.

Governing product statement

Scrol-A-Vision is a composition environment for authored traversal: creators arrange narrative change, presentation and distance along a path; readers conduct the actual duration by moving through it.

Contents

1. Purpose and scope
2. Product definition and boundaries
3. What creative software teaches us
4. Principles for the Scrol-A-Vision interface
5. The canonical domain model
6. Studio workspace and GUI specification
7. Authoring workflows
8. Interaction model, history and selection
9. Story package and schema
10. Path allocation and scroll semantics
11. Playback runtime and state evaluation
12. Visual, text and sound channels
13. Validation, diagnostics and production review
14. Application architecture and technology
15. Persistence, assets, publishing and API
16. Testing and acceptance criteria
17. Accessibility, performance and resilience
18. Phased implementation plan
19. Risks, non-goals and unresolved decisions
Appendix A. Grammar-to-software mapping
Appendix B. Example project fragment
References and design precedents

1. Purpose and scope

This document defines the software system required to make the creative grammar usable in practice. The product must help a multidisciplinary creator think in narrative changes and compositional distance, then turn those intentions into a functioning reader experience.

1.1 What the software is

Scrol-A-Vision is both an authoring environment and a playback system. The authoring environment expresses a story in a structured grammar; the playback system resolves that structure into a continuous encounter controlled by reader traversal. Publishing produces a reader-facing work that does not require the authoring interface.

1.2 What it is not

- Not simply a web page with animation triggers attached to scroll events.
- Not a video editor whose primary unit is elapsed seconds on a fixed timeline.
- Not a conventional comic-layout tool whose principal object is a sequence of static panels.
- Not a game engine by default: scrolling does not automatically imply branching, simulation or consequential plot choices.
- Not a generative author replacing the creator's judgement. Validation can find missing or conflicting data; editorial meaning remains human work.

1.3 Normative language

Word	Meaning in this blueprint
MUST	Required for a conforming baseline implementation or published project.
SHOULD	Recommended default. A documented reason may justify a different choice.
MAY	Optional capability that must not be assumed by every work.
EDITORIAL	A human judgement prompt; not an automated pass/fail condition.

1.4 Separation of standard and implementation

The grammar's hierarchy and meanings are canonical. UI layout, data structures, API choices, storage and rendering algorithms are implementation decisions. This blueprint proposes them in enough detail to build a coherent first version, while leaving room for the software to evolve.

2. Product definition and boundaries

The initial product should support one authored directed path extremely well before it attempts branching, live collaboration or a full-featured media-editing suite.

2.1 Primary users and their questions

Role	Primary question	Software support
Writer / story architect	What changes, and why does it matter?	Hierarchy, purpose fields, Beat function, entry/exit conditions, notes and validation prompts.
Director / interaction designer	How is that change encountered?	Path allocation, composition cues, camera operations, transitions, preview and reverse traversal.
Illustrator / animator	What appears in the visual field, and how is it framed?	Asset library, layer order, crop/scale/position, viewport preview, asset-to-Beat links.

Role	Primary question	Software support
Sound designer	How does audio reinforce or counterpoint visual change?	Audio lanes, cue attachment to path position, overlap, ducking/fade policy and playback rules.
Reader	Can I control the pace without the interface getting in the way?	A clean Player, responsive path, predictable scroll behavior, accessible controls and optional sound.

2.2 Minimum viable product

MVP means a complete author-to-reader loop for a short story: create the canonical hierarchy, assign a primary function to each Beat, import image assets, define scroll distance and camera states, preview forward and backward, validate structural completeness, save to a portable project package, reopen it and publish a read-only Player version.

2.3 Defer until the core loop is dependable

- Cloud accounts, teams and simultaneous multi-user editing.
- Branching plots and arbitrary interactive state machines.
- Full raster/vector illustration tools or full digital audio workstation features.
- 3D scene authoring, physics and real-time game logic.
- AI-generated story structure or automatic aesthetic scoring.
- Complex analytics or social features.

Product discipline

Import existing art and audio; do not rebuild Photoshop, Krita, Blender or a DAW. Scrol-A-Vision's native craft is the arrangement of meaning and presentation along an authored path.

3. What creative software teaches us

Established creative applications succeed partly because their interfaces are organized around the primary material being manipulated. Scrol-A-Vision should learn from these patterns without copying their assumptions wholesale.

Precedent	Useful interface principle	Adaptation for Scrol-A-Vision	What not to inherit
Photoshop [1]	A dominant canvas, tool/options context and inspectable panels. The current tool and selection determine the useful controls.	Keep the Stage central; use a context-sensitive Inspector; keep asset operations close to the composition.	Do not make the interface primarily a toolbox of effects. A visual effect is subordinate to a narrative purpose.
DAWs / Ableton Live [2]	Arrange multiple tracks against a shared ruler; reuse clips; inspect layers of audio and automation; separate arrangement from performance-oriented views.	Use a Path Editor with aligned visual, text, audio and motif lanes. Shared path coordinates help users see counterpoint and overlap.	Do not make seconds, bars or a playhead the master coordinate. Readers control elapsed time; the authored axis is traversal distance.
Blender [3]	Specialized workspaces for different tasks; editors can be composed in layouts without changing underlying project meaning.	Offer Compose, Direct, Rhythm and Review workspace presets over the same project and selection state.	Do not introduce a mode maze or duplicate state. Workspace switching must not change the story.
Figma [4]	A clear navigation/list area, canvas, and properties panel; selected object determines the properties shown.	Pair the Story Navigator with the Stage and Inspector. Inspect the selected Beat, cue, asset or track using relevant fields.	Do not hide key narrative semantics among dozens of generic visual properties.
Krita [5]	Dockable functional panels and saved workspaces let artists arrange tools around their process.	Allow dock/panel resizing and presets later, while shipping a strong default layout first.	Do not require customization before the application feels usable.

3.1 The distinctive interface object: the authored path

The central editing surface should not be called merely a timeline. A timeline conventionally invites the user to think in elapsed time. Scrol-A-Vision must represent authored traversal: where a change occurs in the path, how much traversal is allocated to it, and what other channels do across the same interval. The Path Editor may look familiar to DAW users, but its ruler is measured in Traversal Units (TU), not seconds.

3.2 A design rule borrowed from all five precedents

Selection, context, shared coordinates

The creator should be able to select a narrative unit in the hierarchy, see its rendered result on the Stage, locate its extent in the Path Editor, and edit its meaning or implementation in the Inspector. These views are projections of one canonical object, not independent copies.

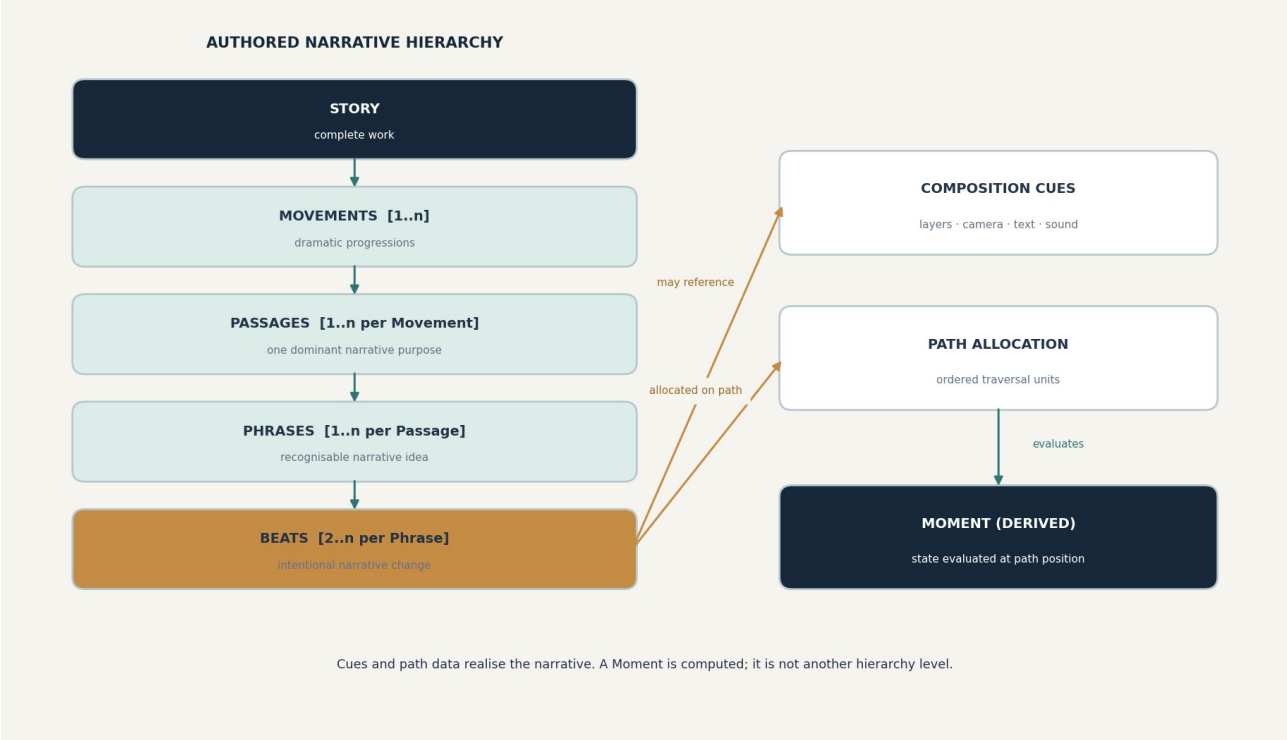
4. Principles for the Scrol-A-Vision interface

The creative standard is not merely displayed in the software; it should shape the workflow and the controls. These principles govern product decisions and should be used when evaluating new features.

Principle	Interface consequence
Meaning before technique	A new Beat begins with purpose and primary function. Camera/effect settings are available, but do not substitute for the narrative intent field.
Distance is authored	The Path Editor allocates relative traversal length, shows gaps and groupings, and warns about unexplained intervals. It never labels TU as time.
Reader duration stays reader-controlled	The default Player is position-driven. It does not assume that a path interval equals a duration in seconds.
One model, multiple views	Navigator, Stage, Path Editor, Inspector, validator and Player read the same Story Model.
Absence is explicit	Pause, Silence and Void can be deliberate cues with typed meaning, rather than blank canvas or accidental missing content.
Narrative and presentation are distinct	REVEAL is a narrative information operation; Approach is a camera operation; Dissolve is a transition method. The UI provides separate fields.
Human judgement is not automated	Structural problems are errors; unsupported playback combinations are warnings/errors; dramatic effectiveness is an editorial prompt.
Author-facing complexity, reader-facing clarity	The internal hierarchy need not become a visible table of contents in the published work.
Reverse traversal is part of the baseline	Scrubbing and reverse scrolling reconstruct a state from position rather than relying on fragile one-way triggers.
Accessible by design	Keyboard operation, reduced motion, clear focus, transcript/caption support and user-controlled audio are part of the feature definition.

5. The canonical domain model

The data model begins with the creative hierarchy and attaches presentation instructions to it. Technical objects must not be mistaken for new narrative levels.



Proposed conceptual model. The hierarchy is authored; composition and path allocation implement it; Moment is derived.

5.1 Canonical hierarchy

Unit	Definition in the product	Baseline constraint
Story	Complete work and root container.	Has stable ID, title, schema version, metadata, settings and one or more Movements when published.
Movement	Major dramatic progression that changes or reframes story condition.	Contains one or more Passages when published; incomplete drafts are allowed.
Passage	Continuous section organised around one dominant narrative purpose.	Contains one or more Phrases; supports optional Entry/Development/Dominant/Exit annotations.
Phrase	Group of at least two Beats expressing one recognisable narrative idea, action or turn.	Contains at least two Beats in a conforming published project.
Beat	Smallest intentional narrative change.	Has one primary function, purpose text, stable ID and a position in the authored path.

5.2 Objects that are not hierarchy levels

- Moment: derived presentation state at a particular path position. Users may bookmark a Moment or attach review comments to it, but the Player computes the state.
- Composition cue: an instruction for a visual layer, camera, text element, sound cue, transition or other channel over a defined path interval.
- Asset: a referenced image, audio file, font or other media item, with metadata and a project-relative path.
- Motif: a stable, named recurring element with references to occurrences. Occurrence identity is structural; whether it develops meaning remains editorial.
- Path segment: a resolved interval in the overall authored path. It is computed from ordered authored traversal allocations and can be cached, not manually copied into each child unit.

5.3 Story state and presentation state

Concept	Stores / represents	Must not be confused with
Narrative intent	What changes, why it matters, what the reader/character may know, and intended resulting condition.	Camera position, opacity or other effect parameters.
Presentation instructions	Layer visibility, framing, scale, transforms, text layout, sound cues and transition method.	The meaning of the Beat itself.
Playback state	Resolved visible/audible scene at current path progress.	A permanent hierarchy node; it is calculated by the runtime.

6. Studio workspace and GUI specification

The interface should feel like a professional creative workstation: one dominant Stage, a visible story structure, an aligned Path Editor and a context-sensitive Inspector. The first release should provide one excellent default workspace before offering extensive docking customization.

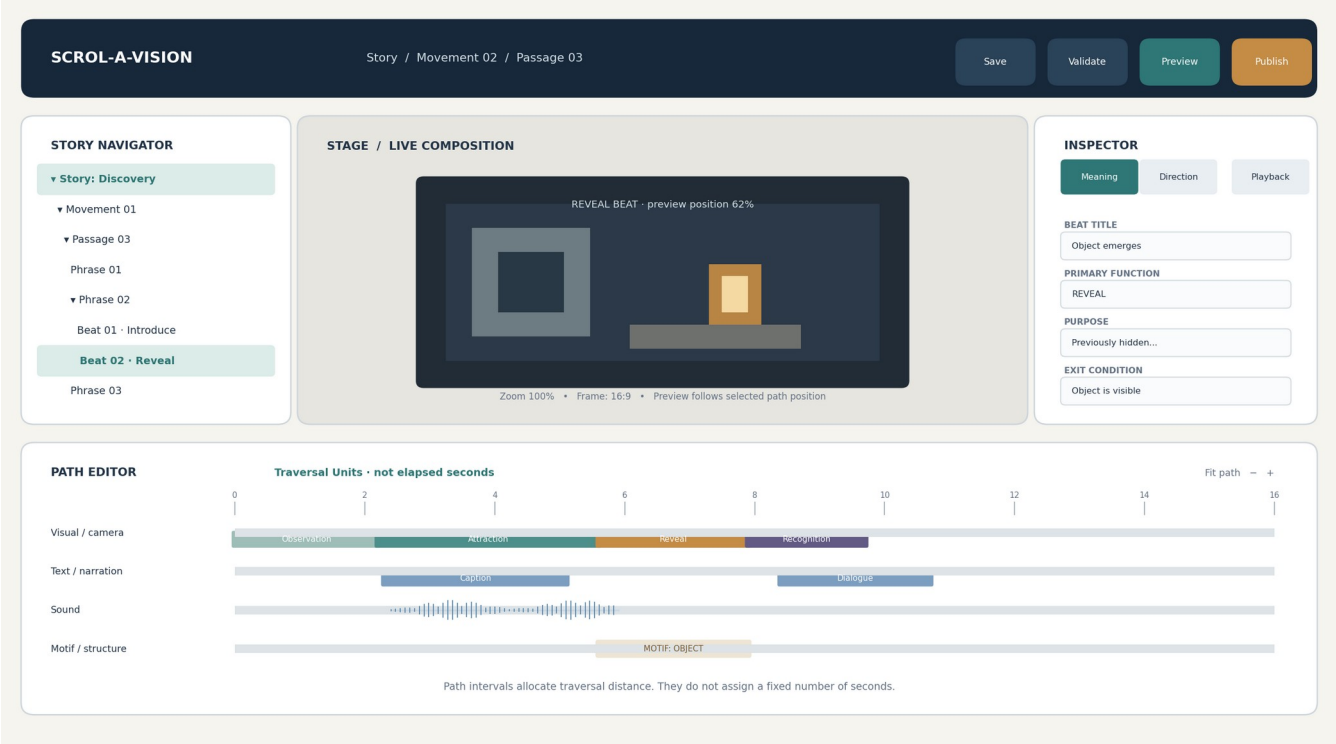


Figure 2. Primary authoring workspace. All areas remain synchronized to one selection and one preview position.

6.1 Global application bar

- Project title and unsaved/saved state. Save status must be visible without being the largest visual element.
- Undo / redo, schema/validation status, Preview, and Publish. Preview and Publish are distinct actions.
- Workspace selector: Compose, Direct, Rhythm, Review. Workspace changes layout emphasis, not project data or playback semantics.
- Project-level commands: new, open, import, export package, preferences, help and version information.

6.2 Story Navigator — left panel

Purpose: navigate the canonical hierarchy. The tree is not a visible page structure and should not imply that readers will encounter the same chaptering. Each row shows unit type, name, status and optional primary Beat function. Supports adding, renaming, duplicating, reordering and nesting only where the hierarchy permits it.

- Drag-reordering updates canonical order; it must not silently change asset identity or authored object IDs.
- Selecting a unit selects its related interval in the Path Editor and updates the Stage/Inspector.
- Empty/incomplete units are permitted in drafts and receive a clear draft status.
- A compact search/filter can later locate units by name, function, motif or validation issue.

6.3 Stage — central composition viewport

Purpose: show the work as the reader will encounter it at the selected path position. It is not a freeform illustration canvas in the MVP; editing is constrained to supported presentation properties and authored assets.

- Displays a stage frame and safe-area guides only while authoring; these guides never publish unless explicitly authored as content.
- Supports zoom-to-fit, 100%, pan when zoomed, background/frame profiles and a clear preview-position indicator.
- Clicking a selectable visual element selects the corresponding layer/cue and updates the Inspector.
- A scrub gesture on the Stage or Path Editor changes preview position; it must not mutate authoring data.
- Provide a mode to preview at a target viewport size; actual reader viewport remains responsive.

6.4 Path Editor — bottom panel

Purpose: compose the spatial rhythm of the work. The horizontal ruler is in Traversal Units or normalized path progress, not seconds. The horizontal extent allocated to a Beat communicates authored distance; it must never imply guaranteed elapsed duration.

Lane	What it displays	Initial editing actions
Structure	Beat/Phrase/Passage spans and boundaries.	Select, resize traversal allocation, reorder where valid, group/ungroup only where hierarchy rules permit.
Visual / camera	Composition cues, layer visibility, camera paths, framing and transition intervals.	Add cue, set endpoints, interpolate supported properties, enable/disable cue.
Text / narration	Captions, dialogue, narration and text appearance/placement cues.	Add text cue, set path interval, line/character placement, visibility and accessibility equivalent.
Sound	Audio cue start/end positions, fades, overlap, one-shot/continuous behavior and mix notes.	Import asset, place cue, set fade/level/policy; actual audio remains user-controllable.
Motif / notes	Motif occurrence markers and editorial annotations.	Link occurrence to motif ID, add comment or reviewer marker.

6.5 Inspector — right panel

The Inspector is context-sensitive but predictable. Selecting a Beat shows its narrative properties; selecting a camera cue shows camera controls; selecting an asset shows asset properties. The Inspector must not hide the narrative function just because visual controls are currently selected.

Inspector tab	Beat selection fields	Cue selection fields
Meaning	Name, purpose, primary function, narrative before/after, secondary notes, editorial question.	Reference to the parent Beat and the intention this cue serves.
Direction	Assets/layers, framing, camera start/end state, transition category, composition notes.	Cue type, target layer/object, start/end values, interpolation choice.
Playback	Traversal allocation, intended relative tempo/dynamic labels, Pause/Silence/Void markers, reverse policy.	Path interval, cue lifecycle, fade/loop/one-shot policy, supported/reduced-motion behavior.

6.6 Workspaces

Workspace	Emphasis	Does not change
Compose	Story Navigator, Meaning Inspector and a compact Path Editor.	Story structure semantics, IDs, path mapping.
Direct	Large Stage, visual layer list, camera/composition controls and path cue editing.	Narrative hierarchy and saved authored path.
Rhythm	Expanded Path Editor with text/audio lanes, interval labels, dynamics and motif markers.	Elapsed-time assumptions; all lanes share traversal coordinates.
Review	Validation panel, editorial checklist, comments/markers and preview controls.	Whether a work is aesthetically “correct”; human review remains human.

7. Authoring workflows

Every common operation should express the grammar in the order a creator naturally thinks: meaning, presentation, path, preview, review.

7.1 Create a Beat

1. Choose the Phrase in the Story Navigator and add a Beat. The system assigns a stable ID and inserts it at an explicit position.
2. Enter a concise purpose and select one primary function from INTRODUCE, CHANGE, REVEAL, EMPHASIZE, RESOLVE or INTERRUPT. Do not preselect a function based on which visual tool will be used.
3. Define the intended narrative change or exit condition; optional secondary effects can be noted without forcing multiple primary functions.
4. Assign an initial traversal allocation. The Path Editor shows where the new Beat sits relative to its neighbours.
5. Add optional assets, visual/camera cues, text and sound; these remain linked to the Beat but may span adjacent Beats when intentionally authored.
6. Preview entry, middle and exit states; inspect the effect of shorter and longer traversal; reverse through the interval.
7. Run structural/playback validation. Editorial prompts remain available but do not block saving a draft.

7.2 Compose a Passage

1. Create or select a Passage and state its dominant narrative purpose.
2. Add Phrases in sequence. Each Phrase should express a recognizable idea, action or turn.
3. Author the Beat sequence and allocate relative traversal distance. Inspect how the empty or low-change intervals relate to adjacent changes.
4. Optionally assign Entry, Development, Dominant and Exit roles to appropriate points/units; these are roles, not extra hierarchy containers.
5. Review the Passage at a fast traversal and at a deliberate traversal, and record where meaning might be lost or overexplained.
6. Validate the production record and make an editorial decision about unresolved conditions.

7.3 Create a reveal

Treat the reveal as narrative information becoming available. Then choose a composition method separately. The same Reveal function might use an Approach, a Cut, a Dissolve, a layer visibility change or a change in text. The Inspector should display this distinction explicitly.

7.4 Create silence or void

Use typed cues, not unexplained blank space. A Void marks an interval of reduced visual information; Silence marks reduced or absent audible material; Pause marks minimal narrative change. These cues may overlap, but they are independently authored and inspected.

7.5 Save and publish

Saving writes the editable project package and preserves authoring metadata. Publishing performs a separate validation pass, produces a compiled/read-only story package and prepares the Player. If publish fails, return actionable diagnostics with a link to the affected unit/cue; do not silently repair creative choices.

8. Interaction model, history and selection

Creative applications feel trustworthy when manipulating the work is predictable. Scrol-A-Vision must preserve object identity and make edits reversible.

8.1 Single selection and shared context

There is one canonical selection state. Selecting a Beat in the Navigator highlights its interval in the Path Editor and focuses the Stage at a default inspect position. Selecting a lane cue focuses its parent Beat and its interval. Preview position is separate from selection: the creator may inspect elsewhere without changing the current selection.

8.2 Editing operations as commands

Represent edits as domain commands (for example, RenameBeat, MoveBeat, SetPrimaryFunction, SetTraversalWeight, AddCameraCue, SetCueEndpoint). A command validates its local constraints, updates the model, creates a history entry and triggers affected projections to refresh.

8.3 Undo / redo

- Undo reverses user-authored mutations, not scroll events or selection changes.
- Commands should store before/after values or deterministic inverses; do not snapshot enormous asset blobs for every edit.
- Undo/redo history is bounded and cleared or branched deliberately when a new project is loaded.
- A schema migration is not represented as dozens of misleading user edits.
- Autosave must not disable undo. Restoring an earlier autosaved project is a separate recovery action.

8.4 Reordering and deletion

Reordering changes sequence order and therefore resolved path positions, but not stable IDs. Deleting a unit must show dependent cues/assets/references and require a confirmable policy: remove dependent cues, reassign them or cancel. Orphaned references should be errors, not silently discarded data.

8.5 Keyboard and pointer conventions

Action	Proposed shortcut / behavior
Save	Ctrl/Cmd+S; show “saving / saved / unsaved / failed”.
Undo / redo	Ctrl/Cmd+Z and Ctrl/Cmd+Shift+Z (platform convention may vary).
Preview / return to Studio	One consistent shortcut; never publish from a shortcut without clear intent.
Navigate path	Arrow/trackpad movement in focused Path Editor; keyboard focus must not be stolen from text inputs.
Rename	F2 or direct inline rename when safe; Enter commits, Escape cancels.
Delete	Delete key only when a selected authored object is focused; confirm destructive hierarchy operations.

9. Story package and schema

The story format is the contract shared by the Studio, validator, playback runtime, file exchange and published Player. It should be versioned, typed, portable and independent of a particular UI layout.

9.1 Format principles

- Store narrative structure in canonical hierarchy order.
- Use stable IDs for authored objects; never derive identity solely from list position or display name.
- Keep creative metadata distinct from computed playback coordinates.
- Use explicit enums for defined grammar terms and namespaced extension fields for future capabilities.
- Do not store absolute viewport pixels as the only camera/position representation.
- Use project-relative asset references; avoid machine-specific paths.
- Support deterministic schema migrations with a backup and a report.

9.2 Proposed project package

```
discovery.savproj/                # ZIP package in distribution; directory form in development
  manifest.json                   # package version, IDs, asset inventory, checksums
  story.json                      # canonical hierarchy and authored composition/path data
  assets/
    images/room.webp
    images/object.webp
    audio/metallic-tone.ogg
  previews/
    cover.webp                    # optional generated preview
  notes/
    production-notes.md           # optional human-readable notes
```

The `.savproj` package is a proposed editable-project container. It is not a final public standard until the MVP, import/export tests and schema migrations demonstrate portability. Published output may use a compiled package with the same semantic content but without editing history or unnecessary authoring metadata.

9.3 Core object types

Type	Required content	Optional extensions
Story	id, schemaVersion, title, movements, asset manifest, project settings.	author credits, description, cover, licence/attribution metadata.
Movement	id, title, passages, purpose/condition note.	dramatic question, reviewer notes, status.
Passage	id, title, phrases, dominant purpose.	entry/development/dominant/exit annotations, relative tempo/dynamic labels.
Phrase	id, title, beats, narrative idea.	notes about turn, motif development or unresolved carry-forward.
Beat	id, title, purpose, primaryFunction, traversalWeight.	before/after conditions, role tags, secondary effects, editorial prompts.
Cue	id, kind, target, start/end path references, typed payload.	easing, blend policy, accessibility alternate, compatibility metadata.
Asset	id, relativePath, mediaType, checksum.	dimensions, duration, attribution, source, usage note.

9.4 IDs and order

Use opaque stable IDs (UUIDv7 or a collision-resistant prefixed ID) for machine identity. Human-readable notation such as S01-M02-P04-H03-B02 may be generated as an optional display label. The “H” segment in older notation should not be assumed to define Phrase identity unless the naming convention is separately standardized. Reordering updates order arrays/path compilation, not object identity.

9.5 Versioning

Store both `schemaVersion` (machine data shape) and `grammarVersion` (creative standard used for authoring). A grammar revision may refine definitions without requiring a file migration; a schema revision may change fields without changing the creative concept. Migrations must be explicit, testable and reversible from a backup.

10. Path allocation and scroll semantics

This is the technical heart of the medium. The software must translate authored spatial pacing into stable progress without pretending that scroll distance dictates elapsed time.

10.1 Definitions

Term	Runtime definition
Traversal Unit (TU)	An abstract unit of authored path length. It is not a second, pixel, musical beat or narrative Beat.
Traversal weight	A positive numeric allocation assigned to an interval. It determines relative path length alongside other intervals.
Path coordinate	A cumulative position along the compiled authored path, represented internally as a normalized scalar from 0 to 1 and optionally as TU.
Reader progress	The current position in the available scroll range, normalized to the active Story path.
Moment	The evaluated presentation state at the current path coordinate.
Viewport profile	A set of rules converting abstract path length into responsive scrollable extent, stage dimensions and asset layout.

10.2 Proposed baseline mapping

Each authored interval carries a positive `traversalWeight`. The path compiler sums the weights in canonical order, calculates cumulative boundaries, and normalizes them to a path domain. The browser adapter maps that domain to actual scroll range according to viewport height and a project-level scale setting. A default profile may treat 1 TU as one viewport-height of scroll traversal; this is a configurable starting convention, not an artistic law.

The reader's `scrollY` is converted to normalized path progress. For a specific interval, the runtime computes local progress from the interval's start/end coordinate. Property-specific interpolators then resolve camera transforms, layer opacity, positions, scales and supported effects. The same input progress must always resolve to the same authored state.

10.3 Segment boundaries and cuts

- A gradual transition occupies an interval and has a defined start and end state.
- A Cut can occur at a boundary and need not consume a meaningful traversal span; its ordering is explicit.
- A Pause may allocate distance while holding narrative state stable.
- A Void may reduce visual information while retaining an authored interval.
- An interval must not be blank solely because a referenced asset failed to load; errors show a diagnostic fallback and do not masquerade as authored Void.

10.4 Resolution, precision and resizing

The compiled path is calculated from abstract weights, never from hand-authored absolute `scrollY` values. On resize, the viewport adapter recomputes physical scroll extent while preserving normalized narrative position where possible. Resizing must not alter the canonical order or authored traversal weights. Floating-point boundary comparisons should use a consistent tolerance and a defined tie-breaking rule (at a boundary, the later segment is active except at the final endpoint).

10.5 No assumed real time

The runtime can observe scroll velocity for optional smoothing or UI feedback, but it must not use velocity to redefine the story's narrative position. An optional autoplay/demo mode may move through the path over time; that is a separate playback mode and must not be treated as the core reading contract.

Critical invariant

Given the same story version, viewport profile and normalized path coordinate, the resolver must produce the same Moment regardless of how the reader arrived there.

11. Playback runtime and state evaluation

The Player should evaluate the current path position, rather than replay a fragile sequence of one-way animation triggers. This is what makes reverse scrolling, seeking and responsive resizing dependable.

11.1 Runtime pipeline

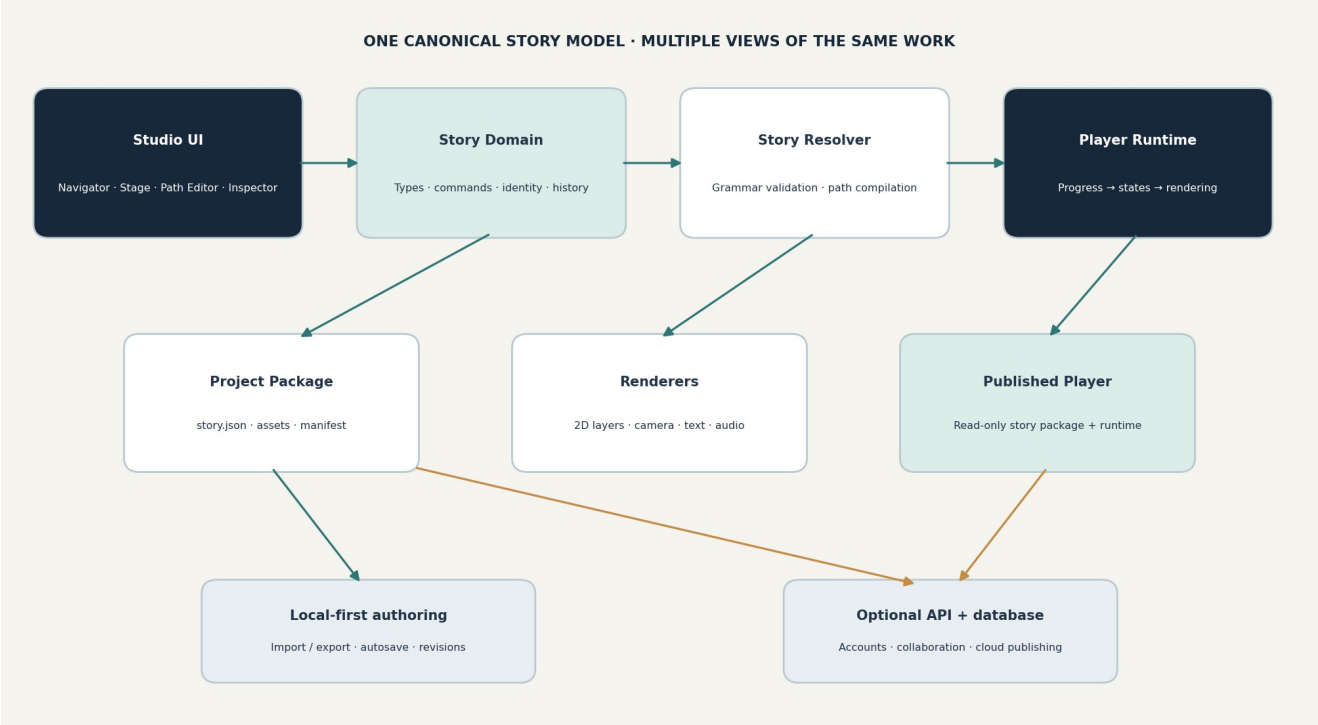


Figure 3. Proposed modular architecture. Cloud services are optional; local authoring and portable packages come first.

11.2 Runtime components

Component	Responsibility	Core invariant
Story Resolver	Validates schema and resolves hierarchy/cues into ordered runtime instructions.	Invalid or unsupported data produces diagnostics rather than unpredictable rendering.
Path Compiler	Calculates intervals and coordinate boundaries from traversal weights and ordering.	Compiled path is derived; authored IDs and weights remain unchanged.
State Evaluator	Computes the visible/audio/text state at a path coordinate.	Pure/deterministic output from project + coordinate + viewport profile.
Renderer	Draws supported image layers, text, camera transform and transitions.	Presentation layer does not decide narrative intent.

Component	Responsibility	Core invariant
Audio Coordinator	Applies cue lifecycle, fade, loop and user sound preferences.	Audio policies are explicit; audio is not allowed to block visual progression.
Viewport Adapter	Maps responsive browser scroll to normalized path progress.	Viewport changes preserve position as closely as possible.
Diagnostics	Reports missing assets, invalid cues, unsupported media and validation issues.	Failure is visible and actionable; missing content is not mistaken for intentional absence.

11.3 Deterministic evaluation

A state-based runtime calculates properties from the current normalized progress. It does not depend on how many scroll callbacks happened, whether the user scrolled rapidly, or whether intermediate frames were rendered. For example, camera scale at progress 0.62 is calculated from its authored curve, not accumulated by repeatedly adding a per-frame delta.

11.4 Interpolation

The baseline should support linear interpolation and a small documented set of easing curves. Each animatable property may use an authored curve when needed. Easing changes how values are distributed over path progress; it does not change the narrative function or the path interval. Advanced splines, procedural animation and 3D camera rigs can be later extensions.

11.5 Performance strategy

- Use requestAnimationFrame or an equivalent frame scheduler to coalesce updates, not one render per raw scroll event.
- Avoid triggering React component-tree rerenders for every pixel of progress; keep high-frequency render state in the runtime/render layer.
- Preload near-future assets and lazily load nonessential distant content; display explicit fallback states for failures.
- Prefer compositor-friendly transforms and opacity for camera/layer motion where possible.
- Measure with representative image-heavy and audio-enabled stories; do not assume a benchmark from an empty demo represents real use.

12. Visual, text and sound channels

The grammar permits several channels, but none should force the others into the same timing model. They relate through the authored path while retaining channel-specific semantics.

12.1 Visual layers and camera

A composition is an ordered set of visual layers and camera/presentation cues. Each layer has a stable ID, asset reference, local transform, visibility rules, accessible description where appropriate and optional motif links. Camera-like movement modifies the reader’s viewport into the composition; it should direct attention and remain separate from a narrative function such as REVEAL.

- MVP visual layers: raster image, SVG where safely supported, text, and simple shape/solid-color layers.
- Layer order controls overlap; order is editable in a dedicated layer stack or Direct workspace.
- Use responsive stage coordinates (normalized or design-space coordinates) instead of storing only device pixels.
- A Splash is a compositional intention; viewport occupation is a rendering result. The two should not be conflated.

12.2 Text

Dialogue, narration, captions, labels and sound effects are authored content with their own placement, visibility and timing/path interval. Text may be attached to a Beat but can span multiple intervals when the story requires it. The Player must preserve legibility across viewport sizes and provide semantic text for accessibility; text should not exist only as pixels inside an image when an equivalent text representation is available.

12.3 Audio

Audio cue position is attached to the authored path, not forced onto a timeline measured in seconds. Audio media itself retains its intrinsic duration, and the cue specifies how it relates to path traversal: play on entry, continue while inside interval, loop ambience, fade according to path progress, or present a one-shot event. The exact behavior must be explicit and reversible-safe.

Audio policy	Meaning	Reverse behavior
Path-faded bed	Volume is derived from current position through an interval.	Volume reconstructs from current position.
Ambient loop	Loop while the reader remains inside a defined range.	Loop eligibility is derived from current range; leaving range fades/stops per policy.
One-shot cue	May trigger on forward entry as a convenience effect.	Must not retrigger on every state evaluation; reverse behavior must be specified (usually no reverse playback).
Narration segment	Audio content associated with a narrative span.	Player must declare whether it pauses/stops on exit, continues until ended, or is optional; no implicit autoplay with sound.
Silence cue	Suppresses or reduces selected audio channels intentionally.	Mix state is derived from position; it is not a missing-audio error.

12.4 Reader sound controls

Sound is optional and must be user-controllable. The published Player should offer an obvious mute/volume control, respect browser autoplay restrictions and save no sensitive behaviour by default. A story must remain intelligible without audio when the creator has authored captions/transcripts or when sound is not essential to comprehension.

12.5 Reduced motion and static alternatives

When reduced motion is requested, the Player should minimize nonessential camera movement and continuous animation while preserving narrative information. Authors should be able to specify static or reduced-motion alternatives for transforms that would otherwise be disorienting. This behavior is a presentation adaptation, not a change to story order or meaning.

13. Validation, diagnostics and production review

Validation enforces structural and technical consistency while preserving the creative standard's explicit respect for human judgement.

13.1 Three validation classes

Class	Examples	Severity / outcome
Structural errors	Missing stable ID; duplicate ID; Phrase contains fewer than two Beats in a published Story; invalid parent relationship; no primary Beat function.	Blocks publish. Draft save remains possible.
Playback errors	Missing required asset; unsupported cue type; invalid interpolation value; negative/zero traversal allocation where the runtime requires positive intervals; broken reference.	Blocks publish when it prevents reliable playback; otherwise show explicit severity.
Editorial prompts	Purpose field is vague; Passage has no stated exit condition; unexplained long traversal; sound seems unrelated; motif occurs once despite being marked recurring.	Never claim aesthetic failure automatically. Creator can review, annotate and consciously accept.

13.2 Diagnostic record

Each diagnostic SHOULD include a stable code, severity, message, affected object ID, suggested correction and navigable location. The validator should not silently rewrite the story to satisfy a rule. Quick-fix actions may be offered for deterministic data repair, with a preview and undoable command.

```
{
  "code": "PHRASE_MINIMUM_BEATS",
  "severity": "error",
  "objectId": "phrase_01J...",
  "message": "A published Phrase must contain at least two Beats.",
  "suggestion": "Add a Beat or keep this unit as an incomplete draft.",
  "ruleSet": "creative-standard-v1.2"
}
```

13.3 Pre-publish review

- All structural and blocking playback errors resolved.
- Assets present, supported and licensed/attributed as required by the creator's chosen terms.
- Every Beat has one primary function and a purpose statement.
- Every Passage has a dominant purpose and resulting condition, including intentionally unresolved outcomes.
- Authored distance has an intelligible relationship to pacing or attention; suspicious blank spans are reviewed.
- Pauses, Silences, Voids, transitions and motif recurrence are deliberate where marked.
- Reverse traversal, fast traversal and deliberate traversal have been checked.
- Text/audio accessibility and reduced-motion behavior reviewed.

13.4 Conformance versus quality

A conforming package is structurally interpretable and can be played according to its declared instructions. It is not certified as good art. Reviewers can use the creative standard's editorial questions, but the software must not translate those questions into a numerical aesthetic score.

14. Application architecture and technology

The architecture should let the existing prototype mature into a reliable creative engine without a risky rewrite. Start with modular boundaries in one repository; deploy additional services only when a real product need exists.

14.1 Recommended technology

Layer	Recommendation	Reason / boundary
Application UI	React + TypeScript + Vite	Preserves the current working stack; good for an interactive editor and separately built Player.
Domain/schema	TypeScript domain package + Zod (or equivalent runtime schema validator)	Compile-time types do not validate imported/untrusted files; runtime validation is required.
Animation/runtime	Custom deterministic path/state resolver; GSAP/ScrollTrigger only where useful for DOM-level presentation	Do not let a third-party animation library become the canonical source of story progress. Keep progress normalized and seekable.
Rendering	DOM/CSS for the initial 2D layer model; Canvas/WebGL only if profiling demonstrates need	Text/accessibility and ordinary layers are easier in DOM; advanced compositing may need a specialized renderer later.
Local project storage	Portable .savproj file package + browser recovery/autosave	Allows useful authoring without a server and preserves ownership/exportability.
Backend (later)	TypeScript HTTP API (e.g. Fastify) + PostgreSQL	Add when accounts, cloud persistence, team permissions or hosted publication are required.
Testing	Vitest for domain/runtime; Playwright for browser E2E	Tests deterministic logic separately from real browser scroll and UI behavior.

Layer	Recommendation	Reason / boundary
Deployment	Static Player assets behind Caddy; API separately routed when introduced	Fits existing web hosting patterns while isolating the application from unrelated services.

14.2 Module boundaries

```
apps/
  studio/           # React authoring interface
  player/           # reader-facing, read-only runtime
  api/              # later: persistence and publishing API
packages/
  story-schema/     # types, enums, validation, migrations
  story-domain/     # commands, hierarchy, IDs, history
  path-compiler/    # traversal weights -> cumulative coordinates
  playback-runtime/ # state evaluation and cue policies
  renderer-2d/      # layers, camera, text, transitions
  audio-runtime/    # optional audio cue coordinator
  project-package/  # import/export, manifests, asset validation
  validation/       # structural, playback and editorial diagnostics
tests/
  fixtures/         # small versioned sample stories and assets
  e2e/              # browser workflows
```

14.3 Dependency direction

The schema and domain model must not import React components. The Path Compiler consumes validated story data and produces derived path instructions. The runtime consumes compiled instructions and a progress coordinate. UI components issue domain commands and subscribe to derived state. This direction prevents UI details from becoming embedded in story files or runtime semantics.

14.4 Avoid premature microservices

The Studio, Player and API may be separate build targets, but early code can remain in a monorepo and a single API service. Do not create separate services for assets, audio, validation and publishing until scale, deployment isolation or team ownership makes that separation beneficial.

15. Persistence, assets, publishing and API

Portable authoring comes first; cloud features follow a validated local file format. The software should not require a database just to create a story.

15.1 Local-first sequence

1. Implement import/export of a complete project package, including manifest, story data, relative assets and checksums.
2. Add autosave/recovery to a local workspace. Clearly distinguish a recovered session from a saved project file.
3. Test opening the same package on a second machine/path; no absolute local paths should be required.
4. Add hosted storage and accounts only when users need them, keeping the project package as a supported interchange/export artifact.

15.2 Asset management

- Every asset receives a stable project ID and project-relative path.
- Record MIME/media type, checksum, size and optional dimensions/duration/attribution.
- Validate supported formats, missing files and duplicate content without forcing creators to merge distinct asset identities.
- Generated previews/cache files are disposable and must not be the only source of creative data.

- An asset replacement should preserve the stable asset ID where it represents the same logical object, while recording the new checksum/version.

15.3 Hosted data model (later)

Entity	Purpose
users / memberships	Authentication and project permissions when accounts are introduced.
projects	Ownership, display metadata, current revision pointer and access policy.
story_revisions	Immutable versioned story JSON or normalized revisions with a known schema version.
assets	Metadata, checksums and references to object storage; avoid storing large media as relational rows by default.
publications	Published revision ID, public slug, build version, visibility and published-at metadata.
review_comments (optional)	Anchored editorial notes referencing stable unit/cue IDs or a path coordinate plus revision.

15.4 API boundary

The Player should consume a published story manifest and assets; it should not have authoring write access. The Studio talks to project persistence endpoints only after server storage is introduced. All writes must validate schema and permissions server-side; client validation is for user feedback, not a security boundary.

```
Future API outline (not MVP dependency)
GET    /api/projects
POST   /api/projects
GET    /api/projects/{projectId}/revisions/{revisionId}
PUT    /api/projects/{projectId}/draft
POST   /api/projects/{projectId}/validate
POST   /api/projects/{projectId}/publish
GET    /p/{publicSlug}/manifest.json
```

15.5 Publishing contract

Publishing freezes a specific revision, validates all referenced assets and emits a read-only package. The publication must identify the schema/runtime version it targets. An update creates a new revision/publication; it does not mutate history invisibly. A public URL should never expose private draft files or editor-only notes by default.

16. Testing and acceptance criteria

Tests should demonstrate the most important promise of the medium: a creator's authored path produces predictable states, the same state can be revisited, and the story survives editing and publication.

16.1 Test layers

Layer	Test focus	Examples
Schema/domain unit tests	Hierarchy, IDs, commands, migrations and diagnostics.	Phrase minimum, stable IDs after reorder, delete dependency handling, migration round-trip.
Path compiler tests	Cumulative boundaries and normalization.	Unequal weights, exact boundaries, endpoint, zero/invalid weights, many Beats.
Runtime tests	Deterministic state and interpolation.	Same progress yields same camera/layer/audio state when approached forward or backward.
Component tests	Inspector and selection synchronization.	Select Beat -> Stage, Path Editor and Inspector identify the same object.
Browser E2E	Actual scroll, viewport resize, file I/O and publishing.	Create story, add two Beats, set reveal, reverse scroll, save/reopen, publish and open Player.

Layer	Test focus	Examples
Visual regression	Representative viewport and authored story screenshots.	Entry, mid-transition, boundary, Splash, Void and reduced-motion state.

16.2 Acceptance criteria for the first complete release

1. A creator can create a Story with at least one Movement, Passage, Phrase and two Beats using the canonical hierarchy.
2. Every Beat supports one primary function and a narrative purpose separate from camera/transition settings.
3. The Path Editor displays intervals in TU/normalized progress, not seconds, and changing one interval updates derived positions consistently.
4. The Stage resolves its presentation from path position; forward scroll, direct seek and reverse scroll agree at the same normalized coordinate.
5. Missing assets and invalid data produce visible diagnostics instead of being rendered as intentional Void/Silence.
6. Save/export and import/reopen preserve IDs, hierarchy, cues, relative assets and behavior.
7. The Player can render a published revision without exposing editor controls or requiring the Studio build.
8. Validation separates blocking structural/playback errors from editorial prompts.
9. Keyboard-only users can reach principal operations; reduced-motion mode preserves narrative information.
10. Existing prototype behavior and test coverage are not regressed during migration.

16.3 Regression baseline

The current project was previously reported as passing 29 tests and a Vite/TypeScript production build, with browser checks for creating/removing shots, camera/traveler interpolation, exact boundary behavior and reverse scrolling. Treat those as regression obligations: first capture the current suite and test intent, then migrate them to the canonical hierarchy. Do not delete a test only because the data model changes; replace it with an equivalent test of the intended behavior.

17. Accessibility, performance and resilience

A medium based on movement must not assume that every reader can or wants to move in the same way. Accessibility is part of the reading contract and the authoring contract.

17.1 Reader accessibility

- The story must be navigable with keyboard and assistive technologies; visible controls have accessible names and predictable focus order.
- Provide pause/disable motion options when movement is nonessential. Ensure reduced-motion mode does not hide narrative information.
- Avoid forced scroll hijacking, disabled browser navigation or trapping the reader inside the story path.
- Text remains selectable and semantically exposed; offer captions/transcripts and alternative descriptions when appropriate.
- Audio is not assumed to autoplay with sound; reader preferences remain respected.
- Contrast, scaling, focus indicators and usable controls must work on narrow viewports.

17.2 Authoring accessibility

- Hierarchy and cues must be reachable without drag-only interaction; provide keyboard alternatives for reorder and interval resizing.
- Inspector fields have persistent labels, validation messages and understandable units.
- Use labels as well as color to distinguish lane types and diagnostics.
- Warnings link to the affected object and can be dismissed or marked reviewed without hiding blocking errors.

17.3 Performance budgets as test targets

Do not lock exact numbers before profiling the intended asset profile, but define measurable targets during MVP: stable interaction at a representative desktop viewport, no full editor rerender for every scroll pixel, bounded memory while traversing long stories, and graceful loading for large images. Record reference hardware/browser, story size and frame-time measures so performance claims are reproducible.

17.4 Fault tolerance

- A failed asset shows a labeled authoring placeholder and a clear Player fallback; it cannot silently produce an authored-looking blank interval.
- Corrupt project import must leave the current project unchanged and provide a clear error report.
- Autosave failure is visible and offers an export/retry path.
- Schema migration runs on a copy or behind a recoverable backup.
- Unsupported features are explicitly reported at publish time; never quietly drop cues during export.

18. Phased implementation plan

The implementation should be sequenced around technical risk, not feature count. Keep the working prototype and make the grammar executable in small increments.

Phase	Build	Exit condition
0. Establish baseline	Inventory existing code, tests, sample stories, data shapes, current GUI interactions and known defects.	A written baseline and test suite capture existing behavior; no rewrite is assumed.
1. Canonical model	Add versioned schema, stable IDs, canonical Story > Movement > Passage > Phrase > Beat hierarchy, cue types and validation. Build import/export fixture.	A complete test story round-trips through schema validation; UI may still use current editing flow.
2. Path compiler	Implement traversal weights, cumulative boundaries, normalized progress and boundary semantics.	Unit tests prove forward, reverse, direct seeking and resize mapping are deterministic.
3. Stage + Path Editor integration	Synchronize Navigator, Stage, Path Editor and Inspector; migrate existing shot operations into composition cues where needed.	Selection and preview state are consistent across views; legacy behavior passes regression tests.
4. Grammar-aware workflow	Beat purpose/function, Passage purpose, Pause/Silence/Void, dynamics labels, motif links and pre-publish validation.	The Discovery Passage can be fully authored without editing JSON.
5. Project package	Portable .savproj import/export, asset manifest/checksums, autosave/recovery and migrations.	Project opens on a clean environment with all references intact.
6. Independent Player	Read-only Player build, responsive viewport mapping, accessibility/reduced motion and publication build.	A published Discovery story works without Studio code or controls.
7. Hosted persistence	API, PostgreSQL, revisions, auth/permissions, publication slugs and backups if the product needs them.	Server validates writes and published artifacts point to immutable revisions.
8. Advanced creative tools	Camera curve editor, more transitions, audio workflows, review comments, optional branches and collaboration.	Each feature has a real authored use case and does not weaken core determinism.

18.1 First implementation milestone

Use one canonical test work

Implement the “Discovery” Passage from Document A. It needs observation, attraction, reveal, a local Dominant and an Exit. It exercises pacing, hidden information, camera, sound, visual emphasis and resulting condition. Use it as fixture, browser demo, regression test and design review artifact.

18.2 Development rule

Do not build a large option panel before its underlying concept is represented in the schema and resolved by the runtime. Every major control should correspond to a defined authoring value, a deterministic runtime behavior and at least one test fixture. Conversely, do not force a creative concept into the data model merely because a UI control would be easy to add.

19. Risks, non-goals and unresolved decisions

The blueprint is detailed enough to start implementation, but several decisions should be tested against real authored works before becoming irreversible public commitments.

19.1 Primary risks

Risk	Why it matters	Mitigation
Timeline metaphor dominates	Creators may think in seconds and reproduce video-editor assumptions.	Label the ruler Traversal Units; prohibit “duration in seconds” as the core coordinate; test comprehension with creators.
Grammar turns into a form	Too many required fields interrupt creative flow.	Separate required structural fields from optional rich metadata; defer completeness checks to publish where safe.
Over-modeling before test story	The schema becomes abstract and hard to use.	Build the Discovery Passage as an executable fixture at every phase.
Stateful animation breaks reverse	Trigger-only effects drift or retrigger.	Make position-based evaluation default; use explicit entry-event policies only for one-shot cues.
UI mimics existing tools without purpose	Many tracks/panels create complexity without a coherent mental model.	Every panel must answer a grammar question: meaning, presentation, path or review.
Backend too early	Deployment/auth/storage distract from core composition.	Start local-first with a portable package; add cloud capabilities when publishing/users require them.
Overclaiming newness	Vertical scrolling and webcomics already exist.	Position the product as a distinct synthesis and production grammar, not ownership of the act of scrolling.

19.2 Explicit non-goals for v1

- No automatic judgement of emotional impact, pacing quality or narrative coherence.
- No requirement that sound, motion, camera movement or branching be present.
- No requirement for authors to expose the Story hierarchy to readers.
- No fixed conversion of TU into seconds.
- No assumption that authored scroll length maps identically to every screen without a viewport adapter.
- No requirement to replace external illustration/audio tools.
- No claims that all animation must be reversible; rather, exceptions must have declared behavior.

19.3 Decisions to validate with prototypes

- Should the baseline Path Editor display TU only, normalized 0–100% only, or allow both labels simultaneously? Recommendation: show TU with an optional normalized overview.
- Should the default project use one viewport-height per TU? Recommendation: yes as an editable project profile, with a minimum and scale control tested across desktop/mobile.
- Should the Stage use DOM/CSS only in the MVP? Recommendation: begin with DOM/CSS for text/layers and profile before adding Canvas/WebGL.
- How should a one-shot sound behave if the reader jumps across its trigger boundary? Recommendation: document trigger semantics, never retrigger on render, and provide a user preference where appropriate.
- Which author-facing fields are required for publish versus simply encouraged? Test on multiple story styles before finalizing.

Appendix A. Grammar-to-software mapping

This matrix maps the creative standard to the product responsibility. It is a traceability aid: when the standard changes, the corresponding UI, schema, runtime and tests should be reviewed together.

Creative concept in Document A	Studio / schema representation	Runtime or validation behavior
Story → Movement → Passage → Phrase → Beat	Canonical Story Navigator and nested data model.	Validate parent-child relationships; preserve stable IDs.
Beat primary function	One required enum field on each published Beat.	Validator checks presence/value; does not infer aesthetic quality.
Purpose and resulting condition	Narrative fields separate from presentation cue fields.	Available to review and output diagnostics; not rendered automatically unless authored.
Moment	Preview position / bookmark / review marker.	Derived current state from path coordinate.
Authored distance	Positive traversal weight / path allocation.	Path compiler calculates cumulative intervals.
Tempo	Relative editorial label.	Display only; not automatically interpreted as seconds or a speed controller.
Pause	Typed interval/annotation with optional presentation behavior.	Hold narrative changes minimal as defined; do not force audio mute or timer wait.
Silence	Audio-channel cue/annotation.	Apply declared audio mix behavior; distinguish from missing audio assets.
Void	Visual-information cue/annotation.	Render deliberate reduced visual content; asset failures remain diagnostics.
Dynamics	Label or curve attached to a stated scope/channel.	Optional supported property curves; labels alone do not alter rendering.
Dominant	Optional structural role annotation to a unit/position.	Review marker; not automatically treated as whole-story climax.
Camera and transitions	Typed composition cues with separate camera/transition fields.	Interpolate or switch according to cue type; same progress yields same state.
Motif	Stable Motif record and occurrence links.	Show motif map and missing occurrence references; human judges development.
Text and sound counterpoint	Separate lanes aligned on the same traversal coordinate.	Channel-specific playback rules allow overlap, leading/lagging and intentional contrast.
Reader traversal / agency	Player scroll controls path; optional links/detours are separately modelled.	Default sequence remains directed; branching must be explicit.
Creative validation questions	Editorial review panel.	Never convert aesthetic interpretation into automatic pass/fail.

Appendix B. Example project fragment

Illustrative JSON for the intended domain. This is a readable fragment, not yet a published JSON Schema. Exact field names and required/optional status should be frozen only after the first implementation fixtures and migration tests pass.

```
{
  "schemaVersion": "1.0.0", "grammarVersion": "1.2",
  "id": "story_discovery", "title": "Discovery",
  "pathProfile": {"unit": "TU", "viewportHeightsPerTU": 1.0, "scale": 1.0},
  "movements": [{
    "id": "movement_unknown", "title": "The Unknown",
    "purpose": "Turn an ordinary space into a source of uncertainty.",
    "passages": [{
      "id": "passage_discovery", "title": "Discovery",
      "dominantPurpose": "Reveal an object before its meaning is understood.",
      "phrases": [{
        "id": "phrase_attraction", "title": "Attraction",
        "narrativeIdea": "A sound draws attention toward the corner.",
        "beats": [
          {"id": "beat_observe", "title": "Observe room",
           "purpose": "Establish the room and its emptiness.",
           "primaryFunction": "INTRODUCE", "traversalWeight": 2.0,
           "compositionCueIds": ["cue_room"]},
          {"id": "beat_reveal", "title": "Object appears",
           "purpose": "Make the hidden object visible.",
           "primaryFunction": "REVEAL", "traversalWeight": 1.25,
           "compositionCueIds": ["cue_approach", "cue_tone"]}
        ]
      }]
    }]
  }],
  "compositionCues": [
    {"id": "cue_approach", "kind": "cameraTransform",
     "targetLayerId": "layer_object", "relativeStart": 0.0, "relativeEnd": 1.0,
     "from": {"x": 0, "y": 0, "scale": 1.0},
     "to": {"x": 0.18, "y": 0.12, "scale": 1.8}, "easing": "easeInOut"},
    {"id": "cue_tone", "kind": "audio", "assetId": "asset_tone",
     "policy": "oneShotOnForwardEntry", "reversePolicy": "doNotRetrigger"}
  ]
}
```

Notes on the example

- The path compiler derives cumulative interval positions from the ordered Beat traversal weights. The JSON does not duplicate calculated absolute path coordinates.
- The camera cue expresses presentation; the Beat's REVEAL function expresses narrative meaning.
- The audio policy is explicit because one-shot sound cannot be safely treated as a pure continuous property in the same way as camera scale.
- The fragment omits required manifest details, asset declarations and schema-level constraints; those belong in the formal schema and complete fixture.
- The example shows a practical modeling direction, not a final immutable schema.

References and design precedents

These sources inform interface patterns discussed in Section 3. They are precedent references, not endorsements and not instructions to reproduce another application's complete design. The application-specific recommendations in this blueprint are product-design decisions proposed for Scrol-A-Vision.

[1] Adobe. "Photoshop workspace overview." Adobe Help, last updated 9 June 2026. Describes the application bar, panels, tools panel, document window, options bar and contextual task bar.

https://helpx.adobe.com/ca/photoshop/desktop/get-started/learn-the-basics/workspace-overview.html

[2] Ableton. "Arrangement View." Live Reference Manual, version 12. Describes arrangement overview, ruler, lanes and linear composition; used as a reference for synchronized tracks, with traversal rather than seconds as Scrol-A-Vision's master axis.

https://www.ableton.com/en/manual/arrangement-view/

[3] Blender Foundation. "Workspaces" and "Areas." Blender Manual. Describes task-oriented workspace layouts and interchangeable editors.

https://docs.blender.org/manual/en/latest/interface/window_system/workspaces.html

[4] Figma. "Design, prototype, and explore layer properties in the right sidebar." Figma Learn. Describes navigation, canvas, toolbar and context-dependent properties.

https://help.figma.com/hc/en-us/articles/360039832014-Design-prototype-and-explore-layer-properties-in-the-right-sidebar

[5] Krita Foundation. "Navigation" / "Dockers." Krita Manual 5.3.0. Describes dockable functional panels and saved workspaces.

https://docs.krita.org/en/user_manual/getting_started/navigation.html

Primary creative source. "Scroll-Driven Storytelling Grammar — The Creative Standard," Document A, v1.2, October 2026. Internal project standard supplied for this work.

Implementation status

This document is a proposed blueprint. Before calling the file format stable, freeze a JSON Schema, add at least one complete project fixture, implement import/export and deterministic path evaluation, and run the acceptance tests in Section 16. Changes that alter the meaning of creative terms should be considered against Document A and recorded in both documents as appropriate.

Closing principle

Build around the authored path, not around effects. The author says what changes and why it matters; the Studio helps direct how it is encountered; the Player makes that intention available to a reader who controls the pace.